

A Unified Sequential Equivalence Checking Approach to Verify High-Level Functionality and Protocol Specification Implementations in RTL Designs

Carlos Ivan Castro Marquez, Marius Strum, Wang Jiang Chau

Department of Electronic Systems
School of Engineering, University of São Paulo
São Paulo, Brazil
{cicastrom, strum, jcwang}@lme.usp.br

Abstract—Formal techniques provide exhaustive design verification, but computational margins have an important negative impact on its efficiency. Sequential equivalence checking is an effective approach, but traditionally it has been only applied between circuit descriptions with one-to-one correspondence for states. Applying it between RTL descriptions and high-level reference models requires removing signals, variables and states exclusive of the RTL description so as to comply with the state correspondence restriction. In this paper, we extend a previous formal methodology for RTL verification with high-level models, to check also the signals and protocol implemented in the RTL design. This protocol implementation is compared formally to a description captured from the specification. Thus, we can prove thoroughly the sequential behavior of a design under verification.

Keywords—*Sequential equivalence checking; high-level models; communication protocol; sequence of states; RTL design; formal verification.*

I. INTRODUCTION

In functional verification, the correctness of the RTL description (the design under verification, DUV) is checked against the system specification, which can be represented eventually by a functionally equivalent model. Simulation-based techniques make use of either the so-called golden models that describe the expected circuit behavior at a higher level of abstraction or sets of assertions added to the DUV's descriptive code. The main advantage of simulation-based verification is its ability to spot quickly easy-to-find bugs, and in fact, it is the most widespread technique for commercial designs [1]. Nevertheless, cycle-accurate simulation is still far from being efficient enough to provide a basis for exhaustive verification practices. Different techniques have been proposed to circumvent such fact [2, 3], but, although productivity can be improved, simulation-based verification state coverage remains reasonably limited.

Formal methods have always been the alternative to simulation whenever verification exhaustiveness is an issue. Model checking is one of the most widely used formal methods, whereby one checks whether the conditions

expressed by a set of properties are fulfilled at every state, or transition, for a finite state machine (FSM) that represents the DUV. Simplifications are adopted, as in [4, 5], in order to manage the complexity in two central points of the model checking approach: to seek a comprehensive set of properties and the time each relevant property, added to match FSM behavior, needs to be checked.

Instead of a set of properties, formal sequential equivalence checking (SEC) may adopt a reference model to be compared to the functionalities implemented in the DUV. The greatest part of techniques for SEC has been developed to check consistency of circuit descriptions up to the RTL abstraction level, usually, aimed to verify the consistency of gate/transistor-level netlists [6]. Besides the associated high computational complexity, they do not support checking at electronic system level (ESL).

A few works have contributed to the equivalence checking between ESL and RTL descriptions. Extensions of SEC have been proposed in [7-9], where information and/or code are added to the reference model in order to make it comparable to the RTL representation, which, actually, leads to added complexity in the equivalence checking process. As a form to reduce equivalence engine effort, similarities between the FSMs of the high level (HL) and RTL models can also be exploited by simplifying the latter; since only the functional aspects must be tested, the DUV is stripped of some RTL-specific constructs, drawing it up to the reference model representation level, as proposed in [10-12]. In this way, the number of objects to be compared in the SEC is smaller, making the computation lighter; on the other hand, the behavior of signals specific to the RTL model must be verified at a later step.

In [10], sequences of events of two FSMs are checked; in order to do so, FIFOs and other high-level constructs are used to wrap the RTL description and, thus, abstract it up to the reference model's level. Therefore, in fact, such approach reduces the potential advantage in performing SEC on descriptions that are more abstract. In [11], state sequences consist of short paths between each two FSM control points,

This work was partially supported by the National Council of Technological and Scientific Development, CNPq, from Brazil.

which makes the number of paths very large. The method has been devised for a high-level scheduling tool environment and is not of general application.

A more general approach was proposed in [12], for two FSM descriptions that follow the same algorithm. In such work, a FSM with data (FSMD) that represents the RTL description is reduced through the elimination of inputs and outputs, which are specific to this model, and not present in the reference model. These signals and associated variables are not considered in the verification process and only the functional characteristics described in the reference model are checked. The reason is that, traditionally, the sequential equivalence checking has been only applied between two circuit descriptions having a one-to-one correspondence for states, as well as for memory elements.

In this paper, we propose a complete methodology for verification of RTL code by comparing it with a behavioral high-level description. The approach from [12] is extended to include, in the same theoretical framework, the equivalence checking of the signaling protocol implemented exclusively in the RTL representation. The RTL protocol implementation is formally compared to a description captured from the system specification. In this manner, we are able to prove the functional coherence for an overall sequential behavior of a design under verification.

II. SEQUENTIAL EQUIVALENCE CHECKING

For the case of two FSM having identical sets of input and output alphabets, with a bijective (or at least surjective) relation between their states, their equivalence can be determined by analyzing directly the equivalence between their states. We formalize this, as follows.

Definition 1. A FSM, M , is defined as the 6-tuple $M = \{S, I, O, \lambda, \delta, s_0\}$, where:

- $S = \{s_0, \dots, s_r\}$ is a finite set of $r+1$ states, so that $r \in \mathbb{N}$;
- $I = \{v_{i0}, \dots, v_{in}\}$ is the input alphabet, composed by a set of $n+1$ variables;
- $O = \{v_{o0}, \dots, v_{om}\}$ is the output alphabet, composed by a set of $m+1$ variables;
- $\delta: S \times I \rightarrow S$ is the function of transition between states;
- $\lambda: S \times I \rightarrow O$ is the output variable update function;
- s_0 is the initial state.

Definition 2. Given two FSMs, M_1 and M_2 , such that $I_1 = I_2 = I$ and $O_1 = O_2 = O$, two states $s_1 \in S_1$ and $s_2 \in S_2$ are equivalent if, and only if, starting from these states, any finite sequence of input conditions $\{i_0, i_1, \dots, i_n\}$, with variables $\{v_{i0}, v_{i1}, \dots, v_{in}\} \in I$, results in the same sequence of output assignments $\{o_0, o_1, \dots, o_n\}$, with variables $\{v_{o0}, v_{o1}, \dots, v_{on}\} \in O$, when applied to either M_1 or M_2 ; it is assumed that the sequence of inputs is recognizable by both machines from the states s_1 , or s_2 .

Definition 3. Two FSMs, M_1 and M_2 , such that $I_1 = I_2$ and $O_1 = O_2$, are said to be equivalent if their initial states, s_{01} and

s_{02} , are equivalent. Otherwise, the machines are called distinguishable.

The concepts above can be applied to check the equivalence between machines that represent circuits described at the same level of abstraction. However, some issues and restrictions must be considered when making use of high-level descriptions as reference models for formal SEC of RTL designs. These are presented in the next section.

III. VERIFICATION OF RTL DESIGNS WITH RESPECT TO HL SPECIFICATIONS

By and large, the behavior of RTL designs can be divided in computation (i.e., treatment of data and control described in the HL reference model) and communication (i.e., interface signals and protocol described only in the DUV). Illustrated in Figure 2, this is a scenario that can be found in typical hardware design flows, where a HL functional specification is refined, and then, a protocol specification, in terms of interface signals, is included or adapted to obtain a RTL description. The result is a hardware core that is more complex than either the HL or protocol specification individually, by including characteristics from both parts, besides other auxiliary elements (signals, registers, etc.).

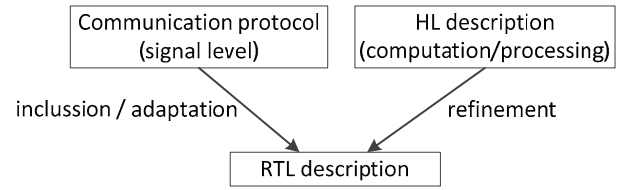


Fig. 2. Inclusion of computational and communication aspects in RTL designs.

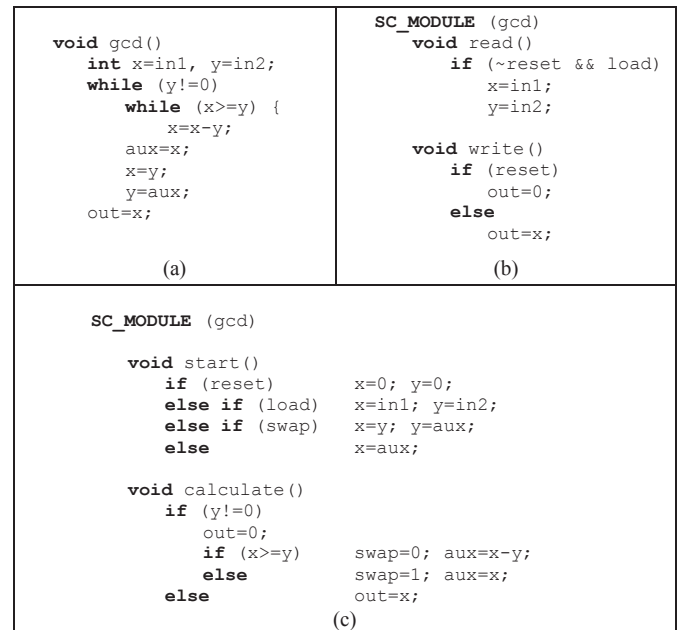


Fig. 1. SystemC-like GCD calculator. (a) HL model, (b) protocol specification, (c) RTL core.

An example of a GCD calculator RTL design is given in Figure 1c, in SystemC pseudocode. It can be seen that the RTL code includes signal *aux* from the HL model (Figure 1a) and signals *reset* and *load* from the protocol specification. Also, it contains the signal *swap*, which was described at neither of these.

In this work, we are proposing a methodology to verify formally both computational and protocol aspects in the RTL description. Thus, the rest of the paper considers three machines, M_{HL} , M_P , and M_{RTL} , in such way that M_{RTL} must comply with the behavior of both M_{HL} (which represents the computation) and M_P (which represents interface signals and protocol) for the DUV to be correct. Our method is to follow the same principle of traditional SEC, as defined above, i.e., M_{RTL} must generate the same output sequences as M_{HL} and M_P , for the same sequences of inputs. Nevertheless, the following issues must be taken into account.

- M_{RTL} may contain a greater number of variables, which means that its alphabet is different from that of M_{HL} and M_P .
- States may be found at either M_{HL} or M_P that have no equivalent state in M_{RTL} , and vice versa.
- Inputs and outputs may appear even following the same order at both M_{HL} and M_{RTL} (or M_P and M_{RTL}), but occurring at different points in time.

As in [12], such problems can be overridden by checking complete sequences of states, instead of individual states as in traditional SEC. The following section provides a complete formalism for the verification of both computation and protocol aspects in the RTL implementation.

IV. FORMAL CONCEPTS FOR RTL VS. HL VERIFICATION

A. Considerations for discrepancies between alphabets

Let us denote as I_{HL} and O_{HL} , I_P and O_P , I_{RTL} and O_{RTL} , the sets of input and output variables of M_{HL} , M_P , M_{RTL} , respectively. From now on, and whenever possible, the set V refers to the union $I \cup O$; thus, for the sake of simplicity, it is possible to make reference to the sets V_{HL} , V_P , and V_{RTL} .

For the present work, it will be always the case that $V_{RTL} \supseteq V_{HL}$ and $V_{RTL} \supseteq V_P$, which is coherent with the statements from previous section. From this, it is then possible to define the set $V_{\cap} = V_{\cap HL} \cup V_{\cap P}$ as all the variables that M_{RTL} shares with M_{HL} and M_P (note that $V_{\cap HL} = V_{HL}$ and $V_{\cap P} = V_P$). Finally, the set $V'_{RTL} = V_{RTL} \setminus V_{\cap}$, corresponds to those variables that appear exclusively as elements from M_{RTL} . This is illustrated in Fig. 3

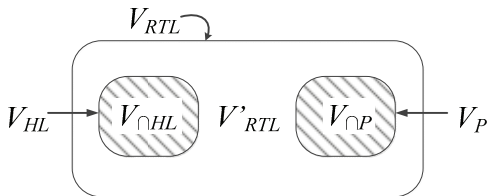


Fig. 3. Relations between sets of variables belonging to M_{HL} , M_P , and M_{RTL} .

B. Sequences of states for state and event disparity

As mentioned earlier, states may have no one-to-one relation between M_{RTL} and M_{HL} or between M_{RTL} and M_P . Also,

events may occur at different points in time. The following definitions establish the basis to solve the problem of applying SEC under such situation.

Definition 4. Given a FSM $M = \{S, I, O, \delta, \lambda, s_0\}$, a sequence of states is defined as the set $k = \{s_0, s_1, \dots, s_{n-1}, s_n\}$, $k \in S$, so that $\delta(s_0, i_0) = s_1, \dots, \delta(s_{n-1}, i_{n-1}) = s_n$, with $n \geq 1$. This, for any input sequence $\{i_0, \dots, i_{n-1}\}$, with variables $\{v_0, \dots, v_{n-1}\} \in I$, that trigger the corresponding transitions.

Thus, any machine M can be decomposed in a set of sequences $K = \{k_0, \dots, k_m\}$, with $m \geq 1$. Under the present framework, any sequence $k \in K$ must obey the following conditions:

- k begins at s_0 , and is finite ($n < \infty$);
- Let $s_i \in S$ be a state with more than one outgoing transitions, so that the choice between those transitions is taken upon the value of a variable $v_i \in I$. More than one instance of s_i may appear in k iff:
 - There is a path of states that leaves from, and returns to s_i .
 - There is a change in the value of v_i through such path, so that, when returning to s_i , the control flow may continue through a different transition.

If the previous condition is false, then each transition leaving s_i derives in a different sequence of states.

A brief example of the strategy above, for extracting the set K from a machine M , is illustrated in Fig. 4As shown, the variable v_i has initially a value that triggers the transition from s_i to s_x . Then, the variable is assigned a value that, when reaching s_i again, triggers the transition from s_i to s_w . The resulting sequence is $k = \{s_0, s_i, s_x, s_y, s_i, s_w, s_n\}$. Should the assignment “ $v_i=0$ ” not exist, then the machine would be decomposed in the sequences $k_0 = \{s_0, s_i, s_x, s_y\}$ and $k_1 = \{s_0, s_i, s_w, s_n\}$.

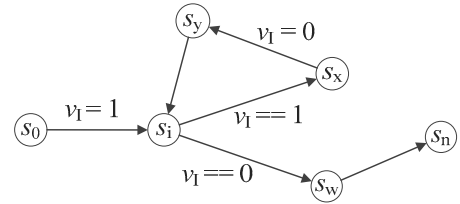


Fig. 4. Decomposition of a FSM in a single sequence of states.

Definition 5. Given a sequence of states, k_{RTL} , the reduction function, $R(k_{RTL}): V_{RTL} \rightarrow V_{\cap}$, is defined as a set of operations that reduce the variables in k_{RTL} only to those belonging to $V_{\cap}$, that is, the result of $R(k_{RTL})$ is a sequence k^R_{RTL} , at which $V'_{RTL} = \{\emptyset\}$. The operations performed by R are constant propagation, removal, and grouping.

1) Constant propagation. The values of variables belonging to V_{RTL} , which have data interdependencies with variables belonging to $V_{\cap}$, are propagated to these. The resulting sequence is named k^1_{RTL} .

2) Removal. All inputs and outputs whose variables belong to V'_{RTL} are removed from k^1_{RTL} , and replaced by the symbol ‘ $\emptyset$ ’. The resulting sequence is named k^2_{RTL} .

3) Grouping. Let s_i and $q=\{s_{i+1}, \dots, s_n\}$ be a state and a subsequence of states, respectively, contained in k_{RTL}^2 , such that s_i precedes q immediately. Given the conditions: a) the input of s_i , i_i , is not unspecified; b) the inputs of the states in q are unspecified, then, s_i and q are grouped, forming a new single state $s_{(i...n)}$, whose output transition is set to be triggered by the input specified in s_i , producing the outputs specified in q , and pointing to the same state, s_{n+1} , as pointed by s_n . Idle states, i.e., with no input and output specified, are grouped, indifferently, with the state immediately preceding, or succeeding. The resulting sequence is named k_{RTL}^R .

An example of the reduction operation is shown in Fig. 5. In this case, $V_\cap = \{a, b, x, y\}$ and $V'_{RTL} = \{d, e, f, u, v, w\}$. At k_{RTL}^1 , all values from variables of V'_{RTL} have been propagated to those of $V_\cap$. At k_{RTL}^2 , all inputs and outputs whose variables belong to V'_{RTL} have been removed, and, finally, all possible groupings have been performed to obtain k_{RTL}^R .

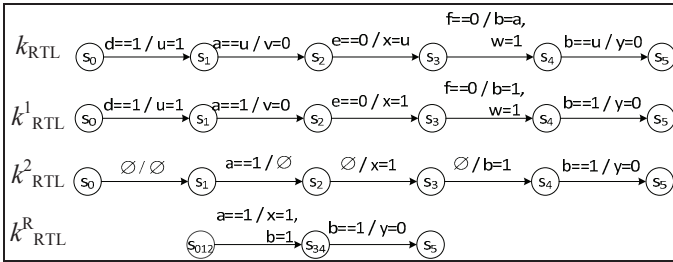


Fig. 5. Example of reduction of a sequence of states.

Next, we state the conditions for k_{RTL} , k_{RTL}^1 , k_{RTL}^2 and k_{RTL}^R to show the same behavior with respect to $V_\cap$.

Postulate 1. Two sequences of states, k_1 and k_2 , have the same behavior with respect to a set of variables, V , if the following conditions are satisfied.

- 1) k_1 and k_2 have equal sets of input variables, with the same ordering.
- 2) k_1 and k_2 have equal sets of output variables, and k_2 is a retracted sequence of k_1 .
- 3) If i_a and i_b are two inputs, and o an output, with variables $\{v_{ia}, v_{ib}\} \in I$ and $v_o \in O$, respectively, then i_a triggers the same output assignment, o , at both k_1 and k_2 , be it directly, or indirectly. For the sake of clarity, an output, o , is a direct result of an input, i_a , if it occurs in the same state as i_a , or indirect, if it occurs at a subsequent state, but before any other state where a new input, i_b , appears.

It can be proved that the application of any operation from Definition 5 on sequences of states do not affect the original behavior of the sequence (Postulate 1) with respect to $V_\cap$.

Theorem 1. The operations of the function R do not alter the behavior between k_{RTL} and k_{RTL}^1 , k_{RTL}^1 and k_{RTL}^2 , or k_{RTL}^2 and k_{RTL}^R , with respect to $V_\cap$.

It is not difficult to demonstrate this theorem for each condition of Postulate 1, but that will be omitted, due to the lack of space in this text. Considering that the property stated in Postulate 1 is transitive, which can be proved easily, then the following corollary treats the complete reduction operation.

Corollary 1. Any sequence k_{RTL}^R presents the same behavior of a sequence k_{RTL} , with respect to the set of variables $V_\cap$, as long as k_{RTL}^R is obtained by applying the reduction function, R , on k_{RTL} , i.e., $R(k_{RTL})=k_{RTL}^R$.

C. HL Sequence Expansion

It can be observed that sequences from a RTL description, after applying the reduction operation, show the same set of symbols from $V_\cap$, allowing them to be compared. Besides that, each grouped RTL sequence presents a valid input condition in every state, therefore, for purposes of easing the comparison of sequences, the same is assumed to the sequences from M_{HL} and M_P . In fact, it does not imply any loss of generality, since, in case the assumption is not true, a grouping operation, as stated in Postulate 1, may be applied in order to correct it.

The concepts on equivalence just presented for state sequences can be extended to two machines, since they are composed by sets of sequences. The functional equivalence between M_{RTL} and M_{HL} , or M_P , can be determined as long as there is a k_{RTL}^R that describes the same behavior for every k_{HL} , or k_P , with respect to $V_\cap$. Nevertheless, such perspective can only be considered a “weak” verification because there is no means to be assured that, in case there is no match between sequences in M_{RTL} and M_{HL} , or M_P , it means they are not equivalent. Ideally, there should be warranty of absence of false negative verification results, but this is very complex if applied to any two M_{RTL} and M_{HL} (or M_P). The present framework includes a “stronger” verification strategy, achieved by restricting the whole set of all possible k_{RTL} . Such a simplified set of sequences is defined as “RTL extended sequences”.

Definition 6. Let $k_{HL} \in K_{HL}$ be a sequence with n states and $k_{RTL} \in K_{RTL}$ another one with m states, such that $m \geq n$. Under the assumption that the following conditions are true, then k_{RTL} is named an extended sequence of k_{HL} .

- 1) Besides the inputs and outputs with variables belonging to V'_{RTL} , k_{RTL} contains, maintaining order and quantity, all the inputs and outputs with variables belonging to $V_\cap$, that are also contained by k_{HL} .
- 2) For each state $s_{HL} \in k_{HL}$, and each input i_{HL} , with $v_{iHL} \in I_\cap$, exists a state $s_{RTL} \in k_{RTL}$, and an input i_{RTL} , with $v_{iRTL} \in I_\cap$, such that $i_{HL}=i_{RTL}$, and the result of $\lambda(s_{HL}, i_{HL})$, when defined, produces the same result as $\lambda(s_{RTL}, i_{RTL})$ directly or indirectly.

An example with three sequences k_{HL} , k_{RTL1} , e k_{RTL2} , with $I_\cap = \{v_{i1}, v_{i2}\}$, $O_\cap = \{v_{o1}, v_{o2}\}$, $I'_{RTL} = \{v_{i3}\}$ e $O'_{RTL} = \{v_{o3}, v_{o4}\}$, is shown in Fig. 4. The sequence k_{RTL1} is an extended sequence of k_{HL} , as every output occurs as result of the same input at both sequences. Note that in k_{RTL1} the second instance of o_2 is an indirect result of $\lambda(s_{22}, i_2)$. On the contrary, k_{RTL2} is not an extended sequence of k_{HL} because the second instance of o_2 is a direct result of the input $i_1 \in I_\cap$.

V. METHODOLOGY AND IMPLEMENTATION

A tool was implemented following the theory from previous sections. It consists of a Python script core that integrates third-party software. Its general workflow is shown in Fig. 6

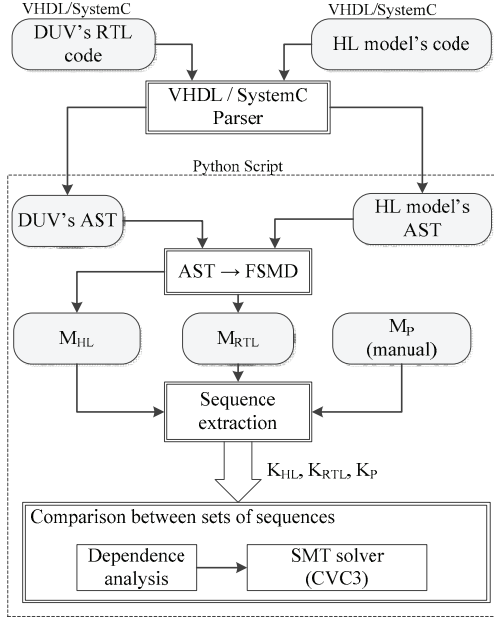


Fig. 6. Overall verification methodology.

Initially, the tool receives and parses the HDL files for the HL model and the DUV; both VHDL and SystemC are accepted. The parser throws the corresponding abstract syntactic trees, ASTs, which are converted into FSM with data (FSMD) representations. The FSMD for the protocol must be written manually under the present framework, by studying the design's functional specification. Next, the sets of sequences K_{HL} , K_P , and K_{RTL} are extracted. Finally, the equivalence checking between K_{HL} and K_{RTL} , and between K_P and K_{RTL} , is performed. This last step is divided in two tasks: dependence analysis, and solving. The first task checks if there are inputs or outputs, along a sequence of states in K_{RTL} , that may occur in different order than the one specified by a sequence in K_{HL} , or K_P . Finally, the second task traverses and compares the sets of sequences by applying the SMT solver CVC3 [14]. The blocks performing sequence extraction and comparison follow the algorithms presented in [12].

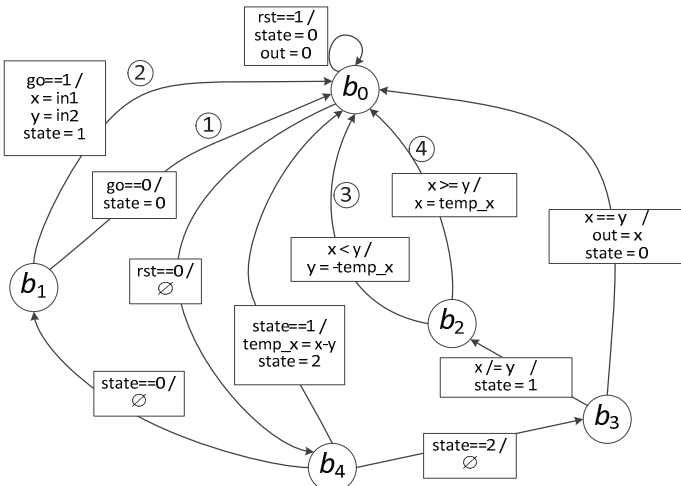


Fig. 7. FSMD representation of GCD example.

VI. APPLICATION EXAMPLE

For illustration purposes, we show how our approach is applied to a GCD calculator, initially described in VHDL. Given that FSMD generation for the HL and RTL descriptions is automatic, only the protocol verification is presented, with the resulting FSMD is shown in Figure 7. In order to obtain the protocol reference FSMD, the functional specification must be analyzed. For this example, a timing diagram that includes all signal relations for the correct operation of the circuit is used (Figure 8). By observing the diagram, it is possible to establish the following conditions.

- 1) signal *go* must be asserted after a *rst* pulse;
- 2) input ports can be read only if *go* is '1';
- 3) output port *out* can be written only if *go* is '1';
- 4) signal *rst* must be '0' throughout computation, and if asserted, *out* must be set to '0'.

The previous conditions can then be synthesized in the FSMD, M_P , of Figure; **Error! No se encuentra el origen de la referencia.** from which, in turn, the sequences from Figure **Error! No se encuentra el origen de la referencia.** can be extracted. The sets of variables for this machine are $V_{\cap E} = \{rst, go, in1, in2, out\}$, and $V' = \{state, x, y, temp_x\}$. If the DUV implements correctly the protocol specification, the K_{RTL} must contain one extended sequence of k_{P0} , and one of k_{P1} . In this example we show one sequence, k_{RTL} (there are more in K_{RTL}), that is an extended sequence of k_{P1} . Figure **Error! No se encuentra el origen de la referencia.** illustrates such sequence, and how, after applying the reduction function on it, the result is a sequence equivalent to k_{P1} .

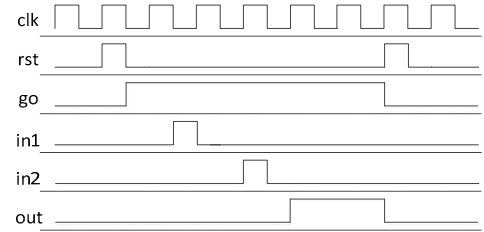


Fig. 8. Protocol timing diagram for the GCD calculator.

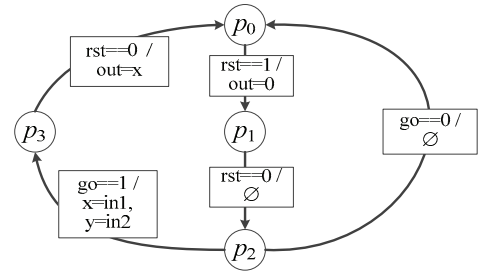


Fig. 9. Protocol FSMD, M_P .

VII. RESULTS

The tool from Section V was applied on a first set of descriptions in VHDL. Except for the GCD module, which was manually written, these modules were obtained in RTL from OpenCores [15]. For these study cases, only their correctness with respect to the protocol specification was tested (the computational aspects verification appears in [12]). The

FSMDs for each protocol were constructed by analyzing each specification document. The results for the VHDL cases appear summarized in Table 1. Column 1 shows the DUV examples, which were a CRC, a GCD, a CORDIC processor in polar to rectangular and rectangular to polar conversion modes, a MDCT, and an AES algorithm. Columns 2 and 3 show the number of states that resulted for each M_P and M_{RTL} , respectively. Columns 4 and 5 present the total number of sequences of states that were extracted for each M_P and M_{RTL} , respectively. The total number of branches, i.e., more than one transition leaving a single state, appear in columns 6 and 7 for the machines M_P and M_{RTL} , respectively. The last column shows the runtime for each case, from the moment the AST is read until the solver finishes, and it is possible to observe that the tool conducts the complete checking with reasonable efficiency.

k_{P0}		k_{P1}	
i_{P0}	o_{P0}	i_{P0}	o_{P0}
$rst == 1$	$out = 0$	$rst == 1$	$out = 0$
$rst == 0$	$\emptyset$	$rst == 0$	$\emptyset$
$go == 0$	$\emptyset$	$go == 1$	$x = in1, y = in2$
		$rst == 0$	$out = x$

(a)

k_{RTL}		k_{RTL}^R	
i_{RTL1}	o_{RTL1}	i_{RTL1}	o_{RTL1}
$rst == 1$	$state = 0, out = 0$	$rst == 1$	$out = 0$
$rst == 0$	$\emptyset$	$rst == 0$	$\emptyset$
$state == 0$	$\emptyset$	$go == 1$	$x = in1, y = in2$
$go == 1$	$x = in1, y = in2, state = 1$	$rst == 0$	$out = x$
$rst == 0$	$\emptyset$		
$state == 1$	$temp\ x = x - y, state = 2$		
$rst == 0$	$\emptyset$		
$state == 2$	$\emptyset$		
$x == y$	$out = x, state = 0$		

(b)

Fig. 10. Sequences of states obtained for M_P (a), and equivalent sequence found for k_{P1} before and after reduction (b)

Table 1. Results of checking between M_P and M_{RTL} for VHDL cores.

DUV	States in M_P	States in M_{RTL}	Sequences in M_P	Sequences in M_{RTL}	Branches in M_P	Branches in M_{RTL}	Time (ms)
crc	5	5	2	7	2	23	9.8
gcd	4	5	2	4	2	11	14.5
cordic: p2r	6	28	1	193	2	25	45.9
cordic: r2p	6	49	1	251	2	37	38.1
mdct	15	36	2	167	8	76	15.6
aes	11	265	3	256	5	124	36.2

The second set of tests consisted on applying the tool on SystemC examples, which are submodules that process bit streams at baseband level in a Bluetooth core implementation, (an in-house design). These experiments focused on testing the tool's ability to discover design bugs injected on purpose to each M_{RTL} . The results are shown in Table 2. The first column shows each DUV example, while columns 2 and 3 show the number of states for each M_{HL} and M_{RTL} , respectively. Columns 4 and 5 show the number of branches for each

machine, and columns 6 and 7 the number of bugs injected and discovered in M_{RTL} , respectively. Our tool implementation cannot find all bugs in a single run; it was necessary to execute several checks, and correct the bug found, iteratively, until no more errors were found in M_{RTL} . Column 8 in Table 2 shows the number of runs until the tool could not detect any bugs further. For the CRC, FEC, and FHEC, not all bugs could be detected, which was due to a mistaken initial use of the CVC3 solver. This was corrected and the tool was able to find all errors injected. For this case, the runtimes also demonstrate the tool's efficiency.

Table 2. Results of checking between M_{HL} and M_{RTL} for SystemC cores.

DUV	States in M_{HL}	States in M_{RTL}	Branches in M_{HL}	Branches in M_{RTL}	Bugs injected in M_{RTL}	Bugs detected in M_{RTL}	Number of runs	Mean time (ms)
crc	25	29	49	57	50	45	50	89.3
fec	43	47	70	78	50	48	50	65.4
fhec	34	42	65	73	50	45	50	81.0
white	36	44	44	52	50	50	26	56.9
cipher	21	29	24	32	50	50	33	25.3

REFERENCES

- [1] Talupur, M., "Hardware model checking: Status, challenges, and opportunities," in *Formal Methods in Computer-Aided Design (FMCAD)*, 2011, p. 154.
- [2] E. Romero, M. Strum, W. Chau, "Manipulation of Training Sets for Improving Data Mining Coverage-Driven Verification". *Journal of Electronic Testing* 29: 223-236, 2013.
- [3] B. Mammo, D. Chatterjee, D. Pidan, A. Nahir, A. Ziv, R. Morad, and V. Bertacco, "Approximating checkers for simulation acceleration," in *Design, Automation & Test in Europe*, pp. 153-158, 2012.
- [4] H.M. Le, D. Grosse, and R. Drechsler, "Towards analyzing functional coverage in SystemC TLM property checking," in *International Conference on High Level Design Validation and Test Workshop*, 2010, pp. 67-74.
- [5] T. K. Nguyen; J. Sun, Y. Liu, and J. S. Dong, "A model checking framework for hierarchical systems," in *International Conference on Automated Software Engineering*, 2011, pp. 633-636.
- [6] P. Molitor, and J. Mohnke, *Equivalence Checking of Digital Circuits: Fundamentals, Principles, Methods*, Kluwer, 2004.
- [7] 18. SLEC Family Product Overview. Calypto. <http://calypto.com/products/slec/overview>. Accessed 20 August 2013.
- [8] A. Koelbl, R. Jacoby, H. Jain, and C. Pixley, "Solver technology for system-level to RTL equivalence checking," in *Design, Automation & Test in Europe*, 2009, pp.196-201.
- [9] K. Hao, F. Xie, S. Ray, and J. Yang, "Optimizing equivalence checking for behavioral synthesis," In *Design, Automation and Test in Europe*, 2010, pp. 1500-1505.
- [10] N. Bombieri, F. Fummi, G. Pravaddelli, and J. Marques-Silva, "Towards Equivalence Checking Between TLM and RTL Models," in *International Conference on Formal Methods and Models for Codesign*, pp. 113-122, 2007.
- [11] C. Lee, C. Shih, J. Huang, and J. Jou, "Equivalence checking of scheduling with speculative code transformations in high-level synthesis," in *Asia and*
- [12] C. Marquez, M. Strum, W.J. Chau, "Formal equivalence checking between high-level and RTL hardware designs," in 14th Latin American Test Workshop (LATW), pp. 1-6, 2013.
- [13] R. Drechsler, "Advanced Formal Verification," Kluwer Academic Publishers, Norwell, MA, USA, 2004.
- [14] CVC3. <http://www.cs.nyu.edu/acsys/cvc3>. Accessed 15 September 2013.
- [15] OpenCores. <http://opencores.org>. Accessed 20 August 2013.